

**ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ІВАНА ФРАНКА**

**ЗВІТ
ДО ЛАБОРАТОРНОЇ РОБОТИ №2
«Структури даних та функціональна декомпозиція»
з курсу «Професійний Python»
Варіант №1 — «Аналіз успішності студентів»**

Виконав: Лис Артур

Група: ФЕП-31с

Перевірів(ла): _____

Дата: 08.09.2026

Львів — 2026

1. Мета роботи

Навчитися свідомо обирати структуру даних під задачу (list, tuple, set, dict, модуль collections), оцінювати складність основних операцій, писати comprehensions і генераторні вирази, декомпонувати програму на невеликі функції та застосовувати функції вищого порядку: *args, **kwargs, lambda, closures і decorators. Результат — модуль обробки структурованих даних, який стає частиною наскрізного проєкту «Система обліку студентів».

2. Індивідуальний варіант і постановка задачі

Варіант №1 — «Аналіз успішності студентів». Вхідні дані: ПІБ, група, дисципліна, оцінка. Потрібно реалізувати список усіх студентів, унікальні групи, середній бал, найкращого студента, студентів із балом вище заданого, групування за групами, рейтинг, кількість студентів у кожній групі та індекс за прізвищем.

Обов'язкові елементи: list, tuple, set, dict, структура з collections, list/set/dict comprehensions, щонайменше 6 функцій, функції з *args і **kwargs, lambda, closure, decorator, сортування, фільтрація, групування, агрегація, пошук, аналіз складності трьох операцій і benchmark. Із завдань підвищеної складності реалізовано: універсальну функцію фільтрації, універсальне сортування, параметризований decorator, closure для створення фільтрів, вкладене групування, статистику через Counter, deque для історії останніх операцій, benchmark list проти dict та функцію pipeline.

3. Структура проєкту

Модуль реалізовано як підпакет analytics у проєкті лабораторної роботи №1, а не як окремий проєкт: методичні вказівки рекомендують розвивати один репозиторій протягом курсу. Пакет підключено до точки входу прапорцем --analytics, а також запускається окремо командою `python -m student_manager.analytics`. Репозиторій: <https://github.com/ArturLys/student-manager>.

```
student_manager/
├── src/
│   └── student_manager/
│       ├── analytics/
│       │   ├── __init__.py
│       │   ├── __main__.py
│       │   ├── analytics.py
│       │   ├── benchmark.py
│       │   ├── data.py
│       │   ├── decorators.py
│       │   └── processors.py
│       ├── __init__.py
│       ├── cli.py
│       ├── config.py
│       ├── exceptions.py
│       ├── main.py
│       ├── models.py
│       └── registry.py
└── tests/
    └── test_analytics.py
```

```
|   └─ test_registry.py
|   └─ .gitignore
|   └─ pyproject.toml
|   └─ README.md
```

4. Вхідні дані та вибір структур

Одиниця даних — запис «студент, група, дисципліна, оцінка». Для нього обрано `NamedTuple GradeRecord`: це справжній `tuple` (незмінний, `hashable`, придатний для `set` і як ключ `dict`), але з іменованими полями замість «магічних» індексів. Усі записи зберігаються в `list`, бо це впорядкована послідовність, яку зручно фільтрувати й сортувати. Перелік дисциплін — `tuple`, бо він фіксований. Унікальні групи та дисципліни отримуються через `set`. Для індексу «студент → його записи», групування та середніх балів використано `dict`.

З модуля `collections` застосовано три структури: `defaultdict(list)` для групування без перевірок на наявність ключа, `Counter` для підрахунку записів у групах (`most_common()` одразу дає найбільшу групу) та `deque(maxlen=5)` як кільцевий буфер останніх викликів декорованих функцій.

Вихідна таблиця задана компактно — по одному кортежу оцінок на студента — і розгортається у плоскі записи одним `list comprehension` з двома циклами:

```
def load_records() -> list[GradeRecord]:
    """Unpack the compact table into flat records: one list comprehension over two loops."""
    return [
        GradeRecord(student, group, discipline, float(grade))
        for student, group, grades in _RAW
        for discipline, grade in zip(DISCIPLINES, grades, strict=True)
    ]
```

5. Опис створених модулів

5.1. data.py

Модель запису, перелік дисциплін, компактна вихідна таблиця та функція `load_records()`, яка повертає 36 записів (9 студентів × 4 дисципліни).

```
class GradeRecord(NamedTuple):
    """Immutable row (ПІБ, група, дисципліна, оцінка). A tuple, so it is hashable and can live in a set."""

    student: str
    group: str
    discipline: str
    grade: float

DISCIPLINES: tuple[str, ...] = ("Професійний Python", "Мова C", "Методи обчислень", "Інженерія даних")

# Compact source table: (студент, група, оцінки в порядку DISCIPLINES).
```

```
_RAW: list[tuple[str, str, tuple[float, ...]]] = [
    ("Гнатишин Марта", "ФЕП-31с", (95, 91, 94, 93)),
    ("Дзюба Остап", "ФЕП-31с", (81, 74, 79, 82)),
    ("Вовк Тарас", "ФЕП-31с", (88, 83, 85, 84)),
    ("Кравець Соломія", "ФЕП-32", (90, 86, 87, 89)),
    ("Пасічник Ірина", "ФЕП-32", (98, 96, 95, 97)),
    ("Мельник Андрій", "ФЕП-32", (77, 70, 72, 75)),
    ("Скиба Юрій", "ФЕП-33", (73, 68, 70, 74)),
    ("Ковальчук Назар", "ФЕП-33", (85, 80, 82, 83)),
    ("Тимків Богдан", "ФЕП-33", (91, 89, 90, 92)),
]
```

5.2. decorators.py

`measure_time` — звичайний decorator: вимірює час виклику через `perf_counter` і друкує його. `log_calls(label, show_result)` — параметризований decorator, тобто фабрика: зовнішня функція замикає `label`, повертає справжній decorator, а той — обгортку. Обгортка додає ім'я функції до `RECENT_CALLS` — deque з `maxlen=5`, тому історія ніколи не росте. `functools.wraps` зберігає ім'я та docstring оригінальної функції. Типи описано через `ParamSpec`, тож туру `--strict` бачить справжню сигнатуру декорованої функції.

```
"""Decorators: timing, call logging with a bounded history of recent operations."""
```

```
from __future__ import annotations
```

```
from collections import deque
from collections.abc import Callable
from functools import wraps
from time import perf_counter
from typing import ParamSpec, TypeVar
```

```
P = ParamSpec("P")
```

```
R = TypeVar("R")
```

```
# Last few decorated calls, oldest dropped automatically: deque(maxlen=...) is O(1) at both ends.
RECENT_CALLS: deque[str] = deque(maxlen=5)
```

```
def measure_time(func: Callable[P, R]) -> Callable[P, R]:
```

```
    """Plain decorator: prints how long one call took."""
```

```
    @wraps(func)
```

```
    def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
```

```
        start = perf_counter()
```

```
        result = func(*args, **kwargs)
```

```
        print(f"[time] {func.__name__}: {(perf_counter() - start) * 1000:.3f} ms")
```

```
        return result
```

```
    return wrapper
```

```
def log_calls(label: str, show_result: bool = False) -> Callable[[Callable[P, R]], Callable[P, R]]:
```

```
    """Parametrised decorator: a factory that closes over `label` and returns the real decorator."""
```

```

def decorator(func: Callable[P, R]) -> Callable[P, R]:
    @wraps(func)
    def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
        result = func(*args, **kwargs)
        RECENT_CALLS.append(func.__name__)
        suffix = f" -> {result!r}" if show_result else ""
        print(f"[{label}] {func.__name__}{suffix}")
        return result

    return wrapper

return decorator

```

5.3. processors.py

Операції рівня структур: унікальні значення (set comprehension), універсальні `filter_items()` і `sort_items()`, які приймають предикат чи ключ, універсальне `group_by()` на `defaultdict`, вкладене групування «група → дисципліна» (dict comprehension поверх групування), індекс за студентом, лінійний пошук для експерименту, `Counter` записів у групах і кількість студентів у групі.

```

"""Structure-level operations on records: filtering, sorting, grouping, indexing, counting."""

```

```

from __future__ import annotations

```

```

from collections import Counter, defaultdict
from collections.abc import Callable, Iterable

```

```

from student_manager.analytics.data import GradeRecord

```

```

Predicate = Callable[[GradeRecord], bool]
KeyFunc = Callable[[GradeRecord], str]

```

```

def unique_groups(records: Iterable[GradeRecord]) -> set[str]:
    """Set comprehension: duplicates vanish on their own."""
    return {r.group for r in records}

```

```

def unique_disciplines(records: Iterable[GradeRecord]) -> set[str]:
    return {r.discipline for r in records}

```

```

def filter_items(records: Iterable[GradeRecord], predicate: Predicate) -> list[GradeRecord]:
    """Universal filter: the caller decides what 'matches' means."""
    return [r for r in records if predicate(r)]

```

```

def sort_items(records: Iterable[GradeRecord], key: KeyFunc | Callable[[GradeRecord], float],
               reverse: bool = False) -> list[GradeRecord]:
    """Universal sort, O(n log n)."""

```

```

return sorted(records, key=key, reverse=reverse)

def group_by(records: Iterable[GradeRecord], key: KeyFunc) -> dict[str, list[GradeRecord]]:
    """Universal grouping. defaultdict spares the 'if key not in dict' dance."""
    groups: defaultdict[str, list[GradeRecord]] = defaultdict(list)
    for r in records:
        groups[key(r)].append(r)
    return dict(groups)

def nested_grouping(records: Iterable[GradeRecord]) -> dict[str, dict[str, list[GradeRecord]]]:
    """Group -> discipline -> records. Dict comprehension over a first-level grouping."""
    return {
        group: group_by(records, lambda r: r.group)
        for group, members in group_by(records, lambda r: r.discipline).items()
    }

def index_by_student(records: Iterable[GradeRecord]) -> dict[str, list[GradeRecord]]:
    """Dict index: O(n) to build, then O(1) average per lookup instead of O(n)."""
    return group_by(records, lambda r: r.student)

def find_linear(records: Iterable[GradeRecord], student: str) -> list[GradeRecord]:
    """The slow way, kept for the complexity experiment: scans everything, O(n)."""
    return [r for r in records if r.student == student]

def count_records_by_group(records: Iterable[GradeRecord]) -> Counter[str]:
    """Counter over a generator expression: no intermediate list."""
    return Counter(r.group for r in records)

def students_per_group(records: Iterable[GradeRecord]) -> dict[str, int]:
    """Dict comprehension; the inner set comprehension de-duplicates students."""
    return {
        group: len({r.student for r in members})
        for group, members in group_by(records, lambda r: r.group).items()
    }

```

5.4. analytics.py

Числовий рівень. `mean_of(*values)` — функція з `*args`, `make_record(**fields)` — з `**kwargs`. `student_averages()` — dict comprehension поверх індексу, декорована `@measure_time`. `best_student()` — `max()` з `lambda`-ключем, `ranking()` — `sorted()` з `lambda`; обидві декоровано `@log_calls`. `make_threshold_filter(minimum)` — closure, `above_threshold()` її використовує. `grade_range()` повертає tuple (min, max), `summary()` збирає агрегати в dict, `pipeline()` приймає `*args` функцій і застосовує їх послідовно.

```

"""Number-level analytics: averages, best student, ranking, thresholds, summaries, pipelines."""

```

```

from __future__ import annotations

from collections.abc import Callable, Iterable
from statistics import mean
from typing import Any

from student_manager.analytics.data import GradeRecord
from student_manager.analytics.decorators import log_calls, measure_time
from student_manager.analytics.processors import group_by, index_by_student, unique_groups

Score = tuple[str, float] # (student, average) – a tuple, because a pair is a pair
ScoreFilter = Callable[[Score], bool]
Step = Callable[[list[Score]], list[Score]]

def mean_of(*values: float) -> float:
    """*args: any number of positional numbers."""
    return mean(values) if values else 0.0

def make_record(**fields: Any) -> GradeRecord:
    """**kwargs: build a record from named fields; NamedTuple validates the names."""
    return GradeRecord(**fields)

@measure_time
def student_averages(records: Iterable[GradeRecord]) -> dict[str, float]:
    """Dict comprehension over the student index; generator expression feeds mean_of via *."""
    return {
        student: mean_of(*(r.grade for r in rows))
        for student, rows in index_by_student(records).items()
    }

def group_averages(records: Iterable[GradeRecord]) -> dict[str, float]:
    return {
        group: mean(r.grade for r in members)
        for group, members in group_by(records, lambda r: r.group).items()
    }

def discipline_averages(records: Iterable[GradeRecord]) -> dict[str, float]:
    return {
        discipline: mean(r.grade for r in members)
        for discipline, members in group_by(records, lambda r: r.discipline).items()
    }

@log_calls("analytics", show_result=True)
def best_student(records: Iterable[GradeRecord]) -> Score:
    """max() with a lambda key over (student, average) pairs."""
    return max(student_averages(records).items(), key=lambda pair: pair[1])

```

```

@log_calls("analytics")
def ranking(records: Iterable[GradeRecord]) -> list[Score]:
    """Sorted by average, best first; ties broken by name thanks to the tuple key."""
    return sorted(student_averages(records).items(), key=lambda pair: (-pair[1], pair[0]))

def make_threshold_filter(minimum: float) -> ScoreFilter:
    """Closure: `predicate` remembers `minimum` after this function has returned."""

    def predicate(score: Score) -> bool:
        return score[1] >= minimum

    return predicate

def above_threshold(records: Iterable[GradeRecord], minimum: float) -> list[Score]:
    is_good_enough = make_threshold_filter(minimum)
    return [score for score in ranking(records) if is_good_enough(score)]

def grade_range(records: Iterable[GradeRecord]) -> tuple[float, float]:
    """(min, max) as a tuple: two values that belong together and never change."""
    grades = [r.grade for r in records]
    return (min(grades), max(grades))

def summary(records: list[GradeRecord]) -> dict[str, Any]:
    """Aggregation in one dict, ready to print or export."""
    averages = student_averages(records)
    low, high = grade_range(records)
    return {
        "records": len(records),
        "students": len(averages),
        "groups": len(unique_groups(records)),
        "mean": mean(averages.values()),
        "min_grade": low,
        "max_grade": high,
        "best": best_student(records),
    }

def pipeline(scores: list[Score], *steps: Step) -> list[Score]:
    """Function pipeline: *args of callables applied left to right."""
    for step in steps:
        scores = step(scores)
    return scores

```

5.5. benchmark.py

Експериментальна частина: генерує 1 000, 10 000 і 100 000 записів, для 50 випадкових студентів вимірює середній час лінійного пошуку в list і пошуку в dict-індексі через timeit, рахує прискорення.


```

"""Experiment: linear search in a list vs lookup in a dict index, 1 000 / 10 000 / 100 000 records."""

from __future__ import annotations

import random
from timeit import timeit

from student_manager.analytics.data import DISCIPLINES, GradeRecord

SIZES: tuple[int, ...] = (1_000, 10_000, 100_000)
LOOKUPS = 50

def make_records(count: int, seed: int = 42) -> list[GradeRecord]:
    rng = random.Random(seed)
    return [
        GradeRecord(f"Студент-{i:06d}", f"ФЕН-{rng.randint(11, 43)}",
                     DISCIPLINES[i % len(DISCIPLINES)], float(rng.randint(60, 100)))
        for i in range(count)
    ]

def find_linear(records: list[GradeRecord], student: str) -> GradeRecord | None:
    for record in records:
        # O(n): every miss walks the whole list
        if record.student == student:
            return record
    return None

def run(sizes: tuple[int, ...] = SIZES, lookups: int = LOOKUPS) -> list[tuple[int, float, float, float]]:
    """Return (n, list µs/lookup, dict µs/lookup, speedup) for every size."""
    rows = []
    for n in sizes:
        records = make_records(n)
        index = {r.student: r for r in records} # dict comprehension, O(n) once
        rng = random.Random(7)
        targets = [records[rng.randrange(n)].student for _ in range(lookups)]
        list_s = timeit(lambda: [find_linear(records, t) for t in targets], number=1) / lookups
        dict_s = timeit(lambda: [index.get(t) for t in targets], number=1) / lookups
        rows.append((n, list_s * 1e6, dict_s * 1e6, list_s / dict_s))
    return rows

def format_table(rows: list[tuple[int, float, float, float]]) -> str:
    lines = [f"{'records':>9} {'list, µs':>12} {'dict, µs':>12} {'speedup':>10}", "-" * 46]
    lines += [f"{'n':>9,} {'list_us:>12.2f} {'dict_us:>12.3f} {'speedup:>9.0f}x" for n, list_us, dict_us,
speedup in rows]
    return "\n".join(lines).replace(",", " ")

if __name__ == "__main__":
    print(format_table(run()))

```

5.6. __main__.py та точка входу

Демонстрація викликає кожну функцію та друкує результат секціями. Фрагмент із pipeline, *args і **kwargs:

```
show("**args, **kwargs")
print(" mean_of(90, 85, 77):", mean_of(90, 85, 77))
print(" make_record(**fields):", make_record(student="Новий Студент", group="ФЕП-31с",
                                             discipline="Мова С", grade=88.0))

show("PIPELINE: рейтинг -> поріг 80 -> перші три")
top = pipeline(
    ranking(records),
    lambda scores: [s for s in scores if make_threshold_filter(80)(s)],
    lambda scores: scores[:3],
)
print_scores(top)
```

До main.py проекту додано прапорець --analytics, який делегує виконання цьому модулю.

5.7. tests/test_analytics.py

Дванадцять pytest-тестів: розмір набору, унікальні групи, середні бали, найкращий студент, порядок рейтингу, closure з порогом, Counter та кількість студентів, вкладене групування, *args/**kwargs, універсальний фільтр і pipeline.

```
import pytest

from student_manager.analytics.analytics import (
    above_threshold,
    best_student,
    group_averages,
    make_record,
    make_threshold_filter,
    mean_of,
    pipeline,
    ranking,
    student_averages,
)
from student_manager.analytics.data import GradeRecord, load_records
from student_manager.analytics.processors import (
    count_records_by_group,
    filter_items,
    index_by_student,
    nested_grouping,
    students_per_group,
    unique_groups,
)

@pytest.fixture
def records() -> list[GradeRecord]:
    return load_records()
```

```

def test_every_student_has_a_grade_per_discipline(records: list[GradeRecord]) -> None:
    assert len(records) == 9 * 4
    assert all(len(rows) == 4 for rows in index_by_student(records).values())

def test_unique_groups(records: list[GradeRecord]) -> None:
    assert unique_groups(records) == {"ФЕП-31с", "ФЕП-32", "ФЕП-33"}

def test_student_average(records: list[GradeRecord]) -> None:
    assert student_averages(records)["Гнатишин Марта"] == pytest.approx(93.25)

def test_best_student(records: list[GradeRecord]) -> None:
    assert best_student(records) == ("Пасічник Ірина", 96.5)

def test_ranking_is_descending(records: list[GradeRecord]) -> None:
    averages = [avg for _, avg in ranking(records)]
    assert averages == sorted(averages, reverse=True)

def test_threshold_closure_remembers_minimum() -> None:
    at_least_90 = make_threshold_filter(90)
    assert at_least_90(("x", 90.0)) and not at_least_90(("y", 89.9))

def test_above_threshold(records: list[GradeRecord]) -> None:
    assert [s for s, _ in above_threshold(records, 90)] == ["Пасічник Ірина", "Гнатишин Марта", "Тимків Богдан"]

def test_counts_and_students_per_group(records: list[GradeRecord]) -> None:
    assert count_records_by_group(records)["ФЕП-32"] == 12
    assert students_per_group(records) == {"ФЕП-31с": 3, "ФЕП-32": 3, "ФЕП-33": 3}

def test_nested_grouping(records: list[GradeRecord]) -> None:
    assert len(nested_grouping(records)["ФЕП-33"]["Мова С"]) == 3

def test_group_averages(records: list[GradeRecord]) -> None:
    assert group_averages(records)["ФЕП-33"] == pytest.approx(977 / 12)

def test_args_and_kwargs() -> None:
    assert mean_of() == 0.0
    assert mean_of(1, 2, 3) == 2.0
    record = make_record(student="А", group="ФЕП-31с", discipline="Мова С", grade=70.0)
    assert record.grade == 70.0

```

```
def test_filter_and_pipeline(records: list[GradeRecord]) -> None:
    python_only = filter_items(records, lambda r: r.discipline == "Професійний Python")
    assert len(python_only) == 9
    top_two = pipeline(ranking(records), lambda s: s[:2])
    assert [name for name, _ in top_two] == ["Пасічник Ірина", "Гнатишин Марта"]
```

6. Використані конструкції

6.1. Comprehensions

```
# list comprehension – плоскі записи з компактної таблиці
[GradeRecord(student, group, discipline, float(grade))
 for student, group, grades in _RAW
 for discipline, grade in zip(DISCIPLINES, grades, strict=True)]

# set comprehension – унікальні групи
{r.group for r in records}

# dict comprehension – індекс student -> record у benchmark
{r.student: r for r in records}

# generator expression – Counter без проміжного списку
Counter(r.group for r in records)
```

6.2. *args та **kwargs

```
def mean_of(*values: float) -> float:
    return mean(values) if values else 0.0

def make_record(**fields: Any) -> GradeRecord:
    return GradeRecord(**fields)

mean_of(90, 85, 77) # 84.0
make_record(student="Новий Студент", group="ФЕП-31c", discipline="Мова C", grade=88.0)
```

6.3. lambda

```
max(student_averages(records).items(), key=lambda pair: pair[1])
sorted(student_averages(records).items(), key=lambda pair: (-pair[1], pair[0]))
filter_items(records, lambda r: r.discipline == "Професійний Python" and r.grade >= 90)
```

lambda тут — короткий ключ або предикат для функції вищого порядку. Уся логіка, що довша за один вираз, винесена у звичайні іменовані функції.

6.4. Closure

```
def make_threshold_filter(minimum: float) -> ScoreFilter:
    """Closure: `predicate` remembers `minimum` after this function has returned."""

    def predicate(score: Score) -> bool:
        return score[1] >= minimum

    return predicate
```

Після повернення з `make_threshold_filter()` локальна змінна `minimum` не зникає: внутрішня функція `predicate` тримає на неї посилання у своєму `__closure__`. Так з однієї фабрики отримуємо скільки завгодно фільтрів із різними порогами.

6.5. Decorator

```
@measure_time
def student_averages(records): ...

@log_calls("analytics", show_result=True)
def best_student(records): ...
```

Перший decorator застосовується напямую. Другий спочатку викликається з параметрами й повертає decorator, тому в ньому одночасно працюють closure (запам'ятований `label`), `*args` і `**kwargs` (прозоре передавання аргументів) та `functools.wraps`.

7. Результат роботи програми

Повний вивід команди `python -m student_manager.analytics` (рядки `[time]` та `[analytics]` друкують decorators):

УСІ ЗАПИСИ (36), перші 8

```
-----
Гнатишин Марта    ФЕП-31с    Професійний Python    95.0
Гнатишин Марта    ФЕП-31с    Мова С                  91.0
Гнатишин Марта    ФЕП-31с    Методи обчислень       94.0
Гнатишин Марта    ФЕП-31с    Інженерія даних        93.0
Дзюба Остап      ФЕП-31с    Професійний Python    81.0
Дзюба Остап      ФЕП-31с    Мова С                  74.0
Дзюба Остап      ФЕП-31с    Методи обчислень       79.0
Дзюба Остап      ФЕП-31с    Інженерія даних        82.0
```

УНІКАЛЬНІ ГРУПИ ТА ДИСЦИПЛІНИ (set)

```
-----
групи:            ['ФЕП-31с', 'ФЕП-32', 'ФЕП-33']
дисципліни:       ['Інженерія даних', 'Методи обчислень', 'Мова С', 'Професійний Python']
```

СЕРЕДНІЙ БАЛ КОЖНОГО СТУДЕНТА (dict, @measure_time)

```
-----
[time] student_averages: 0.226 ms
Гнатишин Марта    93.25
Дзюба Остап      79.00
Вовк Тарас       85.00
Кравець Соломія  88.00
Пасічник Ірина   96.50
Мельник Андрій   73.50
Скиба Юрій       71.25
Ковальчук Назар  82.50
Тимків Богдан    90.50
```

НАЙКРАЩИЙ СТУДЕНТ (max + lambda, @log_calls)

```
-----
[time] student_averages: 0.207 ms
```

```
[analytics] best_student -> ('Пасічник Ірина', 96.5)
Пасічник Ірина – 96.50
```

РЕЙТИНГ (sorted + lambda)

```
-----
[time] student_averages: 0.127 ms
[analytics] ranking
  1. Пасічник Ірина      96.50
  2. Гнатишин Марта     93.25
  3. Тимків Богдан      90.50
  4. Кравець Соломія    88.00
  5. Вовк Тарас         85.00
  6. Ковальчук Назар    82.50
  7. Дзюба Остап        79.00
  8. Мельник Андрій     73.50
  9. Скиба Юрій         71.25
```

СТУДЕНТИ З БАЛОМ >= 85 (closure make_threshold_filter)

```
-----
[time] student_averages: 0.129 ms
[analytics] ranking
  1. Пасічник Ірина      96.50
  2. Гнатишин Марта     93.25
  3. Тимків Богдан      90.50
  4. Кравець Соломія    88.00
  5. Вовк Тарас         85.00
```

СЕРЕДНІЙ БАЛ ГРУП І ДИСЦИПЛІН

```
-----
ФЕП-31с      85.75
ФЕП-32       86.00
ФЕП-33       81.42
Професійний Python  86.44
Мова С       81.89
Методи обчислень   83.78
Інженерія даних    85.44
```

КІЛЬКІСТЬ ЗАПИСІВ І СТУДЕНТІВ У ГРУПАХ (Counter, dict comprehension)

```
-----
ФЕП-31с   записів 12   студентів 3
ФЕП-32    записів 12   студентів 3
ФЕП-33    записів 12   студентів 3
найбільша група: ('ФЕП-31с', 12)
```

ГРУПУВАННЯ (defaultdict) ТА ВКЛАДЕНЕ ГРУПУВАННЯ

```
-----
ФЕП-31с: 12 записів
ФЕП-32: 12 записів
ФЕП-33: 12 записів
ФЕП-32 / Мова С: [('Кравець Соломія', 86.0), ('Пасічник Ірина', 96.0), ('Мельник Андрій', 70.0)]
```

ПОШУК: index (dict) проти лінійного (list)

```
-----
index['Тимків Богдан']: [91.0, 89.0, 90.0, 92.0]
find_linear(...):      [91.0, 89.0, 90.0, 92.0]
```

УНІВЕРСАЛЬНИ filter_items / sort_items (lambda)

```
-----
Пасічник Ірина      ФЕП-32    Професійний Python  98.0
Гнатишин Марта      ФЕП-31с   Професійний Python  95.0
Тимків Богдан        ФЕП-33    Професійний Python  91.0
Кравець Соломія     ФЕП-32    Професійний Python  90.0
```

*args, **kwargs

```
-----
mean_of(90, 85, 77): 84
make_record(**fields): GradeRecord(student='Новий Студент', group='ФЕП-31с', discipline='Мова С',
grade=88.0)
```

PIPELINE: рейтинг -> поріг 80 -> перші три

```
-----
[time] student_averages: 0.138 ms
[analytics] ranking
1. Пасічник Ірина      96.50
2. Гнатишин Марта      93.25
3. Тимків Богдан        90.50
```

ПІДСУМОК (aggregation)

```
-----
[time] student_averages: 0.131 ms
[time] student_averages: 0.120 ms
[analytics] best_student -> ('Пасічник Ірина', 96.5)
records      36
students     9
groups        3
mean         84.38888888888889
min_grade    68.0
max_grade    98.0
best         ('Пасічник Ірина', 96.5)
```

ОСТАННІ ОПЕРАЦІЇ (deque maxlen=5)

```
-----
['best_student', 'ranking', 'ranking', 'ranking', 'best_student']
```

BENCHMARK: лінійний пошук у list проти dict-index

```
-----
records      list  µs      dict  µs      speedup
-----
1 000         31.83      0.152      209x
10 000        381.49      1.136      336x
100 000       3243.81      0.682     4756x
```

8. Аналіз складності операцій

Операція	Структура	Складність	Де в коді
лінійний пошук студента	list	O(n)	processors.find_linear

побудова індексу	dict	$O(n)$, один раз	processors.index_by_student
пошук за ключем	dict	$O(1)$ у середньому	index[student]
перевірка належності	set	$O(1)$ у середньому	group in unique_groups(...)
групування за один прохід	defaultdict	$O(n)$	processors.group_by
підрахунок записів	Counter	$O(n)$	processors.count_records_by_group
сортування рейтингу	list	$O(n \log n)$	analytics.ranking
додавання в історію	deque	$O(1)$	decorators.RECENT_CALLS

Ключова відмінність — пошук. Лінійний прохід по `list` у гіршому випадку торкається кожного запису, тож час зростає разом із даними. Індекс на `dict` коштує одного проходу $O(n)$ під час побудови, але далі кожен пошук — це обчислення хешу й один доступ до комірки, у середньому $O(1)$. Якщо пошуків більше ніж один, індекс окупається одразу. Сортування Timsort має $O(n \log n)$ і його не варто виконувати в циклі — рейтинг обчислюється один раз і далі лише зрізається.

9. Експериментальна частина: benchmark

Для кожного розміру згенеровано записи з унікальними ПІБ, побудовано `dict`-індекс і для 50 випадкових студентів виміряно середній час одного пошуку. Результати запуску на цій машині (Python 3.13, Windows 11):

records	list μ s	dict μ s	speedup
1 000	34.63	0.550	63x
10 000	408.24	0.644	634x
100 000	3351.92	0.732	4579x

Кількість записів	list search, мкс	dict search, мкс	Прискорення
1 000	34.63	0.550	63×
10 000	408.24	0.644	634×
100 000	3351.92	0.732	4579×

Час лінійного пошуку зріс приблизно у 97 разів при зростанні даних у 100 разів, тобто поводить себе лінійно, як і передбачає $O(n)$. Час пошуку в `dict` залишився на рівні часток мікросекунди незалежно від розміру — це $O(1)$. Прискорення на 100 000 записів — близько $4579\times$. Відхилення від ідеально лінійного зростання пояснюються тим, що випадкові цілі пошуку лежать у різних місцях списку (у середньому проглядається половина), а також кешем процесора та накладними витратами інтерпретатора на малих обсягах.

10. Автоматизовані тести та перевірка типів

```
===== test session starts =====
platform win32 -- Python 3.13.0, pytest-9.1.1, pluggy-1.6.0 --
C:\Users\user\vscode\uni\python\project\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\user\vscode\uni\python\project
configfile: pyproject.toml
testpaths: tests
collecting ... collected 21 items

tests/test_analytics.py::test_every_student_has_a_grade_per_discipline PASSED [ 4%]
tests/test_analytics.py::test_unique_groups PASSED [ 9%]
tests/test_analytics.py::test_student_average PASSED [ 14%]
tests/test_analytics.py::test_best_student PASSED [ 19%]
tests/test_analytics.py::test_ranking_is_descending PASSED [ 23%]
tests/test_analytics.py::test_threshold_closure_remembers_minimum PASSED [ 28%]
tests/test_analytics.py::test_above_threshold PASSED [ 33%]
tests/test_analytics.py::test_counts_and_students_per_group PASSED [ 38%]
tests/test_analytics.py::test_nested_grouping PASSED [ 42%]
tests/test_analytics.py::test_group_averages PASSED [ 47%]
tests/test_analytics.py::test_args_and_kwargs PASSED [ 52%]
tests/test_analytics.py::test_filter_and_pipeline PASSED [ 57%]
tests/test_registry.py::test_by_group_ignores_case_and_spaces PASSED [ 61%]
tests/test_registry.py::test_group_average PASSED [ 66%]
tests/test_registry.py::test_best_overall_and_inside_group PASSED [ 71%]
tests/test_registry.py::test_best_of_unknown_group_raises PASSED [ 76%]
tests/test_registry.py::test_grade_outside_scale_is_rejected PASSED [ 80%]
tests/test_registry.py::test_group_name_is_validated PASSED [ 85%]
tests/test_registry.py::test_duplicate_student_is_rejected PASSED [ 90%]
tests/test_registry.py::test_search_combines_criteria PASSED [ 95%]
tests/test_registry.py::test_rating_is_descending PASSED [100%]

===== 21 passed in 0.31s =====

$ python -m mypy src --strict
Success: no issues found in 14 source files
```

11. Робота з Git

Лабораторну роботу №2 додано шістьма комітами поверх історії ЛР1. Репозиторій опубліковано на GitHub: <https://github.com/ArturLys/student-manager>.

11.1. Результат git status

```
On branch main
nothing to commit, working tree clean
```

11.2. Результат git log --oneline

```
9770176 Wire analytics demo into CLI, add tests, bump version to 0.2.0
94e46bd Add list vs dict search benchmark
3810ae2 Add analytics: averages, ranking, threshold closure, pipeline
28f41c3 Add collection processors: filtering, grouping, indexing, counting
5dd0a69 Add timing and logging decorators with a bounded call history
6c8b9a6 Add analytics package with per-discipline grade records
ce3e20c Make stdout re-encoding type-safe for мурі --strict
ca3ca1a Add registry tests and README
c01835d Add console menu and application entry point
8502b7c Add StudentRegistry business logic
7726d17 Add Student model with validation and domain exceptions
9cee0a2 Create project skeleton with src-layout
```

12. Відповіді на контрольні запитання

1. Що таке list?

Впорядкована змінювана послідовність довільних об'єктів; індексується з нуля, підтримує зрізи.

2. Чим list відрізняється від tuple?

list можна змінювати (append, remove, присвоєння за індексом), tuple — ні. tuple компактніший і може бути hashable, якщо всі його елементи hashable.

3. Чому tuple називається immutable?

Після створення не можна змінити ні кількість, ні значення елементів; будь-яка «зміна» створює новий об'єкт.

4. Для чого використовується set?

Зберігати унікальні елементи, швидко перевіряти належність, виконувати об'єднання, перетин і різницю множин.

5. Яка властивість елементів set є важливою?

Hashability: елемент має бути незмінним і мати __hash__, узгоджений з __eq__.

6. Що таке dict?

Асоціативний масив на хеш-таблиці: відображення ключ → значення зі збереженням порядку вставлення.

7. Що таке key-value structure?

Структура, де дані адресуються не позицією, а ключем; кожному ключу відповідає рівно одне значення.

8. Чому пошук key у dict зазвичай швидкий?

Хеш ключа одразу вказує на комірку таблиці, перебирати елементи не потрібно; у середньому $O(1)$.

9. Що таке hashability?

Здатність об'єкта повертати сталий цілочисловий хеш протягом свого життя і коректно порівнюватися на рівність; потрібна для ключів dict та елементів set.

10. Чому list не може бути key у dict?

list змінюваний: якби вміст змінився після вставлення, хеш став би іншим і ключ «загубився б». Тому list не має `__hash__`.

11. Для чого використовується Counter?

Підрахунок кількості входжень елементів; має `most_common()` та арифметику лічильників.

12. Для чого використовується defaultdict?

dict, який сам створює значення для відсутнього ключа (наприклад, порожній list); зручний для групування.

13. Для чого використовується deque?

Двостороння черга з $O(1)$ додаванням і видаленням з обох кінців; з `maxlen` — кільцевий буфер останніх N елементів.

14. Що таке list comprehension?

Компактний вираз `[f(x) for x in xs if умова]`, який будує новий list за один прохід.

15. Що таке set comprehension?

Те саме у фігурних дужках: `{f(x) for x in xs}`; результат — set без дублікатів.

16. Що таке dict comprehension?

`{ключ(x): значення(x) for x in xs}` — будує dict, наприклад індекс за ідентифікатором.

17. Що таке generator expression?

`(f(x) for x in xs)`: лінивий ітератор, який обчислює елементи по одному і не створює проміжний list.

18. Що означає $O(1)$?

Час не залежить від розміру даних: доступ за індексом, пошук ключа в dict у середньому.

19. Що означає $O(n)$?

Час зростає пропорційно кількості елементів: лінійний пошук, повний прохід.

20. Що означає $O(n \log n)$?

Типова складність ефективного сортування (Timsort у Python); лише трохи гірша за лінійну.

21. Що означає $O(n^2)$?

Квадратична: два вкладені цикли по n елементів; у 10 разів більше даних — у 100 разів довше.

22. Яка приблизна складність пошуку у list?

$O(n)$: у гіршому випадку перевіряються всі елементи.

23. Яка середня складність пошуку у set?

$O(1)$ у середньому; $O(n)$ лише у виродженому випадку масових колізій.

24. Яка середня складність пошуку key у dict?

$O(1)$ у середньому.

25. Чому правильна структура даних може прискорити програму?

Бо змінює складність операції: пошук у dict замість list — $O(1)$ замість $O(n)$. У benchmark на 100 000 записів це близько 4579 разів.

26. Що таке функціональна декомпозиція?

Поділ програми на невеликі функції, кожна з яких робить одну річ і має зрозумілу назву.

27. Які переваги малих functions?

Легше читати, тестувати окремо, повторно використовувати й замінювати; менше дублювання.

28. Що таке positional argument?

Аргумент, який зіставляється з параметром за порядком у виклику: $f(1, 2)$.

29. Що таке keyword argument?

Аргумент, переданий за іменем: $f(\text{grade}=90)$; порядок не має значення.

30. Для чого використовується *args?

Щоб функція приймала довільну кількість позиційних аргументів у вигляді tuple.

31. Для чого використовується **kwargs?

Щоб функція приймала довільні іменовані аргументи у вигляді dict.

32. Що таке lambda function?

Анонімна однорядкова функція-вираз: `lambda r: r.grade`.

33. Де доцільно використовувати lambda?

Як короткий key для sorted/max/min або предикат для фільтра, коли окрема іменована функція була б зайвою.

34. Чому складну business logic не варто записувати в lambda?

lambda не має імені, docstring, кількох рядків і важко тестується; для логіки потрібна звичайна функція.

35. Що таке closure?

Внутрішня функція, яка зберігає доступ до змінних зовнішньої функції після її завершення.

36. Які variables може пам'ятати closure?

Вільні змінні з охоплюючих областей видимості, наприклад `minimum` у `make_threshold_filter`.

37. Що таке decorator?

Функція, що приймає функцію й повертає нову (обгортку) з додатковою поведінкою; синтаксис `@name`.

38. Для чого застосовуються decorators?

Логування, вимірювання часу, кешування, перевірка прав, повторні спроби — наскрізна поведінка без зміни тіла функції.

39. Для чого використовується functools.wraps?

Копіює `__name__`, `__doc__` і `__module__` оригінальної функції на обгортку, щоб декорована функція не «втрачала обличчя» у трасуваннях і документації.

40. Чому потрібно оцінювати складність алгоритму?

Щоб передбачити поведінку на великих даних до того, як вони з'являться, і обрати структуру, яка масштабується.

13. Висновок

У роботі до проекту «Система обліку студентів» додано пакет `analytics` — модуль обробки структурованих даних про оцінки за дисциплінами. Дані представлено `NamedTuple`-записами в `list`, унікальні значення — через `set`, індекси, групування та агрегати — через `dict`,

defaultdict, Counter і deque. Реалізовано понад двадцять невеликих функцій, зокрема універсальні фільтр, сортування та групування, функції з *args і **kwargs, closure для порогових фільтрів, звичайний і параметризований decorators та pipeline. Аналіз складності підтверджено експериментом: пошук у dict-індексі на 100 000 записів приблизно у 4579 разів швидший за лінійний пошук у list, а час лінійного пошуку зростає пропорційно кількості записів. Код покрито дванадцятьма тестами і проходить туру --strict; зміни зафіксовано шістьма комітами й опубліковано на GitHub.